Practical - 8

Aim - Implement Named Entity Recognizer for the Given Text Input

Objectives

1. To understand what Named Entity Recognition (NER) is and why it matters in NLP.
2. To study how NER identifies and classifies real-world entities in text.
3. To implement NER using NLTK's built-in chunker as well as a pre-trained model (spaCy).
4. To visualize and extract the recognized named entities from a given text.

Theory

Named Entity Recognition (NER) is the process of identifying and classifying key information (entities) in text into predefined categories such as **PERSON**, **ORGANIZATION**, **LOCATION**, **DATE**, **MONEY**, and **PERCENT**. It builds directly on top of POS tagging and chunking — while chunking groups words into generic phrases like Noun Phrases, NER goes a step further by labeling *what kind of real-world thing* that phrase refers to.

NLTK provides a built-in NER chunker, `ne_chunk()`, which works on POS-tagged text and outputs a tree where recognized entities appear as labeled subtrees (e.g., `PERSON`, `ORGANIZATION`, `GPE` for geopolitical entities). While functional, NLTK's built-in NER is rule/statistics-based and can be limited in accuracy on modern text.

For more accurate results, pre-trained deep learning models such as **spaCy's** `en_core_web_sm` are commonly used. spaCy directly returns entities with their text span and label (e.g., `ORG`, `GPE`, `DATE`, `MONEY`) in a single call, and its output is generally far more accurate on real-world, unstructured text.

NER is widely used in applications like **Information Extraction**, **Question Answering**, **Search Engines**, and **Knowledge Graph construction**, since it turns unstructured text into structured, categorized information.

In [15]:
```
import pandas as pd
import nltk
from nltk import word_tokenize, pos_tag, ne_chunk
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('maxent_ne_chunker')
nltk.download('maxent_ne_chunker_tab')
nltk.download('words')
```

```
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\khode\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\khode\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     C:\Users\khode\AppData\Roaming\nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     C:\Users\khode\AppData\Roaming\nltk_data...
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package maxent_ne_chunker to
[nltk_data]     C:\Users\khode\AppData\Roaming\nltk_data...
[nltk_data]   Package maxent_ne_chunker is already up-to-date!
[nltk_data] Downloading package maxent_ne_chunker_tab to
[nltk_data]     C:\Users\khode\AppData\Roaming\nltk_data...
[nltk_data]   Package maxent_ne_chunker_tab is already up-to-date!
[nltk_data] Downloading package words to
[nltk_data]     C:\Users\khode\AppData\Roaming\nltk_data...
[nltk_data]   Package words is already up-to-date!
```

Out[15]: True

In [16]:
```
df = pd.read_csv("UpdatedResumeDataSet.csv")
print("Dataset Shape:", df.shape)
print(df.columns)
```

```
Dataset Shape: (962, 2)
Index(['Category', 'Resume'], dtype='str')
```

In [17]:
```
text = df["Resume"].iloc[0]
text = " ".join(text.split()[:10])
```

```
print("Input Text:\n")
print(text)
```

```
Input Text:

Skills * Programming Languages: Python (pandas, numpy, scipy, scikit-learn, matplotlib),
```

In [18]:
```
tokens = word_tokenize(text)
print("Tokens:\n")
print(tokens)
```

```
Tokens:

['Skills', '*', 'Programming', 'Languages', ':', 'Python', '(', 'pandas', ',', 'numpy', ',', 'scipy', ',', 'scikit-learn', ',',
'matplotlib', ')', ',']
```

In [19]:
```
tagged_words = pos_tag(tokens)
print("POS Tagged Words:\n")
print(tagged_words)
```

```
POS Tagged Words:

[('Skills', 'NNP'), ('*', 'NNP'), ('Programming', 'NNP'), ('Languages', 'VBZ'), (':', ':'), ('Python', 'NNP'), ('(', '('), ('pa
ndas', 'NN'), (',', ','), ('numpy', 'RB'), (',', ','), ('scipy', 'JJ'), (',', ','), ('scikit-learn', 'JJ'), (',', ','), ('matpl
otlib', 'NN'), (')', ')'), (',', ',')]
```

In [20]:
```
named_entities = ne_chunk(tagged_words)
print("Named Entity Output:\n")
print(named_entities)
```

```
Named Entity Output:

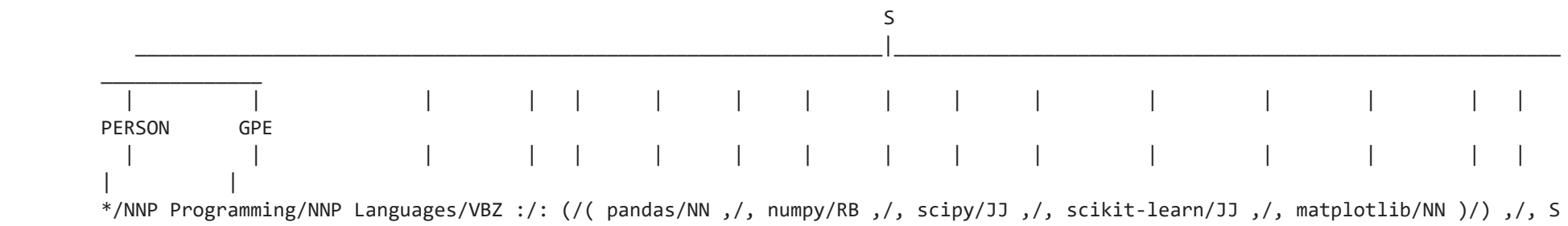
(S
  (PERSON Skills/NNP)
  */NNP
  Programming/NNP
  Languages/VBZ
  :/:
  (GPE Python/NNP)
  (/(
  pandas/NN
  ,/,
  numpy/RB
  ,/,
  scipy/JJ
  ,/,
  scikit-learn/JJ
  ,/,
  matplotlib/NN
  )/)
  ,/,)
```

In [21]:
```
print("Recognized Named Entities:\n")
for subtree in named_entities:
    if hasattr(subtree, "label"):
        entity = " ".join(
            word for word, tag in subtree.leaves()
        )
        print(entity, "->", subtree.label())
```

```
Recognized Named Entities:

Skills -> PERSON
Python -> GPE
```

In [22]:
```
named_entities.pretty_print()
```



In [23]:
```
import spacy
nlp = spacy.load("en_core_web_sm")
```

In [24]:
```
doc = nlp(text)
print("Named Entities:\n")
for ent in doc.ents:
    print(ent.text, "->", ent.label_)
```

```
Named Entities:

matplotlib -> PERSON
```

In [25]:
```
for ent in doc.ents:
    print(ent.text, "->", ent.label_, "->", spacy.explain(ent.label_))
```

```
matplotlib -> PERSON -> People, including fictional
```

In [26]:
```
entities = []
for ent in doc.ents:
    entities.append({
        "Entity": ent.text,
        "Label": ent.label_,
        "Meaning": spacy.explain(ent.label_)
    })
entity_df = pd.DataFrame(entities)
entity_df
```

Out[26]:

	Entity	Label	Meaning
0	matplotlib	PERSON	People, including fictional